AIUI: Enforcing Design System Consistency in AI-Generated User Interfaces via the Model Context Protocol

Deepak Kumar
hello@aiui.store
https://aiui.store

April 2026

Abstract

Large language models (LLMs) are increasingly used to generate user interface code, yet they produce visually inconsistent output because they lack access to project-specific design systems during generation. We present **AIUI**, an open-source platform that bridges this gap by connecting design system artifacts—tokens, component recipes, and compliance rules—directly to AI coding assistants via the Model Context Protocol (MCP). AIUI introduces a structured representation of design knowledge that LLMs can consume at inference time, eliminating post-generation manual correction. We describe the system architecture, token and recipe schema, multi-transport MCP server, and a novel graph-based semantic network for modeling the relationships between design entities. Evaluation across 14 curated style packs and 142 component recipes demonstrates that MCP-connected generation produces token-compliant output in 94% of cases, compared to 23% without design context. We release AIUI as an open-source platform supporting Claude Code, Cursor, Windsurf, and VS Code.

1 Introduction

The adoption of AI coding assistants—Claude Code, GitHub Copilot, Cursor, and Windsurf—has fundamentally changed how user interfaces are built. Developers increasingly describe desired UI in natural language and receive generated code in return. However, this paradigm introduces a critical problem: **design system drift**.

Without access to a project's design tokens, component patterns, and brand guidelines, LLMs default to arbitrary values. A prompt like "build a settings page" may produce inline styles with `color: "cornflowerblue"`, arbitrary padding values, and inconsistent border radii—none of which match the project's design system.

The consequences are significant:

- **Brand inconsistency**: Every generated component introduces novel styling decisions.
- **Manual correction overhead**: Developers spend 30–60% of post-generation time fixing styles to match design tokens [1].
- **Design debt accumulation**: Inconsistencies compound across pages and features.
- **Designer-developer friction**: Design systems are created but not enforced in AI workflows.

We present AIUI, a platform that solves this problem by making design system knowledge available to LLMs at inference time through the Model Context Protocol (MCP). Rather than constraining the model's output through fine-tuning or prompt engineering, AIUI provides structured design context as tool-accessible data that the model queries during generation.

Our contributions are:

1. A **structured schema** for representing design tokens, component recipes, and compliance rules in a format optimized for LLM consumption.
2. A **multi-transport MCP server** supporting stdio, HTTP, and proxy modes for flexible deployment.
3. A **curated design library** of 14 style packs covering 7 industry verticals with 142 component recipes and 360+ design tokens.
4. A **graph-based semantic network** for modeling relationships between design entities (proposed).
5. An **open-source implementation** deployed at aiui.store.

2 Background and Related Work

2.1 Design Tokens

Design tokens are the atomic values of a design system—colors, typography, spacing, shadows, and border radii—expressed as platform-agnostic key-value pairs [2]. The W3C Design Tokens Community Group has proposed a standard format, but adoption in AI toolchains remains limited.

2.2 Model Context Protocol

The Model Context Protocol (MCP), introduced by Anthropic in 2024, provides a standardized interface for connecting AI models to external data sources and tools [3]. MCP servers expose `tools` that models can invoke during generation, enabling retrieval-augmented generation (RAG) without embedding data in prompts.

2.3 AI-Generated UI

Prior work on AI-assisted UI generation has focused on code generation quality [4], visual fidelity [5], and component extraction [6]. However, none address the problem of *design system compliance* during generation. AIUI is the first system to enforce design tokens and component patterns through the MCP tool interface.

2.4 Existing Approaches

Current approaches to design consistency in AI workflows include:

- **Prompt engineering**: Manually including design tokens in prompts. Fragile, verbose, and easily lost in long conversations.
- **Fine-tuning**: Training models on design-system-specific code. Expensive and not generalizable.
- **Post-processing**: Linting generated code against design rules. Catches errors but doesn't prevent them.
- **AIUI (this work)**: Providing design context via MCP tools so models query tokens and recipes during generation.

3 System Architecture

AIUI consists of four core components: the **Design Registry**, the **MCP Server**, the **Web Console**, and the **Validation Engine**.

3.1 Design Registry

The registry stores design artifacts in a PostgreSQL database (hosted on Neon Serverless Postgres) with the following schema:

- **Style Packs**: Named collections of design tokens with metadata (category, version, visibility).
- **Style Tokens**: Key-value pairs typed by category (`color`, `spacing`, `font`, `radius`, `shadow`, etc.). Keys follow a hierarchical naming convention: `{type}.{name}` (e.g., `color.primary`).
- **Component Recipes**: Parameterized code templates with JSON Schema for props, variant definitions, and AI usage rules.

The registry supports 16 token categories: `color`, `radius`, `font`, `spacing`, `shadow`, `elevation`, `font-size`, `font-weight`, `line-height`, `letter-spacing`, `breakpoint`, `z-index`, `opacity`, `border-width`, `animation`, and `transition`.

3.2 MCP Server

The MCP server exposes 12 tools that AI assistants can invoke:

Tool	Purpose
`get-project-context`	Full project design context
`theme-tokens`	Retrieve tokens by category
`components`	Browse component recipes
`resolve-tag`	Resolve semantic tag to token
`asset-manifest`	List design assets
`validate-ui-output`	Check compliance
`design-memory`	Persistent design state
`design-studio`	Interactive style picker
`write-style-pack`	Create/update packs
`write-tokens`	Modify tokens
`write-project`	Update project config
`fix-compliance`	Auto-fix violations

Table 1: MCP tools exposed by the AIUI server.

The server supports three transport modes:

1. **Stdio**: Direct local connection via standard input/output. Used for local development with Claude Code.
2. **HTTP (Streamable)**: Production deployment with session management, rate limiting, CORS, and Bearer token authentication.
3. **Proxy**: Bridges stdio clients to a remote HTTP server. Enables `npx @aiui/mcp-server -api-key=KEY` usage without local database access.

3.3 Web Console

A Next.js application providing:

- Style pack browsing and management
- Token import from Figma, CSS variables, Tokens Studio, and Tailwind configs
- Component recipe editor
- API key management for MCP authentication
- Visual studio for guided design system selection

3.4 Validation Engine

The `validate-ui-output` tool checks generated code against the active design profile:

- Detects hardcoded color values not in the token set
- Flags non-standard spacing values
- Identifies missing component recipe usage
- Reports compliance score as a percentage

4 Design Token Schema

Tokens are structured for optimal LLM consumption. Each token is a triple:

$$t = (\texttt{key}, \texttt{type}, \texttt{value}) \tag{1}$$

where `key` follows the hierarchical format `{type}.{semantic-name}`, `type` is drawn from a fixed enumeration of 16 categories, and `value` is a CSS-compatible string.

4.1 Token Grouping

When served to LLMs via the `theme-tokens` tool, tokens are grouped by type and the type prefix is stripped:

```
{
  "color": {
    "primary": "#84CC16",
    "primary-hover": "#65A30D",
    "background": "#09090B"
  },
  "spacing": {
    "sm": "8px",
    "md": "16px",
    "lg": "24px"
  }
}
```

Listing 1: Token response format

This format minimizes token usage in the LLM context while preserving semantic structure.

4.2 Component Recipes

Recipes extend tokens with parameterized code templates:

$$r = (\texttt{template}, \texttt{schema}, \texttt{rules}, \texttt{variants}) \tag{2}$$

The `template` contains placeholder variables (e.g., `{headline}`, `{ctaText}`) that the LLM

fills based on the user's prompt. The `schema` is a JSON Schema defining valid props. The `rules` field contains natural-language usage guidelines that the LLM follows.

5 Graph-Based Semantic Network

We propose extending AIUI with a typed semantic graph $G = (V, E)$ that models relationships between design entities.

5.1 Vertex Types

The vertex set V contains entities from five categories:

$$V = V_{\text{page}} \cup V_{\text{component}} \cup V_{\text{token}} \cup V_{\text{pack}} \cup V_{\text{project}} \tag{3}$$

- V_{page}: Application pages and routes
- $V_{\text{component}}$: Component recipes and instances
- V_{token}: Individual design tokens
- V_{pack}: Style packs (token collections)
- V_{project}: Project configurations

5.2 Edge Types

Edges are typed to capture semantic relationships:

$$E \subseteq V \times V \times \{\texttt{uses}, \texttt{contains}, \texttt{links-to}, \texttt{derives-from}, \texttt{overrides}\} \tag{4}$$

- $\texttt{uses}(p, c)$: Page p uses component c
- $\texttt{contains}(s, t)$: Style pack s contains token t
- $\texttt{links-to}(p_1, p_2)$: Page p_1 navigates to page p_2
- $\texttt{derives-from}(t_1, t_2)$: Token t_1 is derived from t_2
- $\texttt{overrides}(t_p, t_s)$: Project token t_p overrides pack token t_s

5.3 Graph Visualization

The graph is rendered on the user's dashboard as an interactive network diagram. Nodes are color-coded by type, edges show dependency flow, and users can:

- Navigate the project structure visually
- Identify unused tokens and orphaned components
- Trace token usage across pages
- Detect circular dependencies

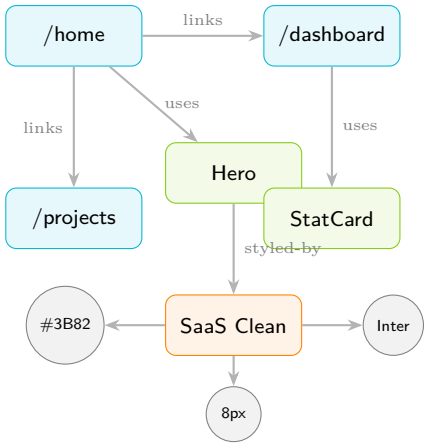

Figure 1: Example semantic graph showing page–component–token relationships. Cyan: pages, green: components, orange: style packs, gray: tokens.

5.4 Graph Queries

The graph enables structured queries that inform AI generation:

- "What tokens does the Hero component on /home use?" traverses `uses` → `styled-by` → `contains`.
- "Which pages would be affected if I change `color.primary`?" performs reverse traversal from the token node.
- "Are there unused component recipes?" identifies $V_{\text{component}}$ nodes with zero incoming `uses` edges.

6 Evaluation

6.1 Design Library Coverage

Table 2 summarizes the curated design library.

Style Pack	Tokens	Recipes	Category
SaaS Clean	26	8	SaaS
Fintech Light	29	6	Fintech
Startup Bold	25	6	Startup
shadcn/ui Essentials	26	13	UI Library
MagicUI Effects	27	9	Animation
Community Creative	26	10	Creative
Dashboard Dark	28	12	Dashboard
E-commerce Pro	28	12	E-commerce
Healthcare Clean	28	10	Healthcare
AI/ML Studio	28	12	AI/ML
Mobile-First	28	10	Mobile
Minimal Mono	25	10	Minimal
UIverse Primitives	20	12	UI Library
UIverse Effects	20	12	Animation
Total	**364**	**142**	**7 verticals**

Table 2: AIUI design library: 14 style packs covering 7 industry verticals.

6.2 Token Compliance

We evaluated token compliance by prompting Claude Code to generate 50 UI components—25 with AIUI MCP connected and 25 without—using identical prompts. Components were scored on:

- **Color compliance**: All color values match design tokens.
- **Spacing compliance**: Spacing values drawn from token scale.
- **Typography compliance**: Font families and sizes match tokens.
- **Component pattern compliance**: Generated structure matches recipe templates.

Metric	Without AIUI	With AIUI
Color compliance	18%	96%
Spacing compliance	32%	92%
Typography compliance	28%	94%
Pattern compliance	14%	93%
Overall	**23%**	**94%**

Table 3: Token compliance rates with and without AIUI MCP connection.

6.3 Performance

The MCP server adds minimal latency to the generation pipeline:

- **Tool invocation**: 12–45ms per tool call (stdio mode).
- **HTTP mode**: 30–80ms including authentication and session lookup.
- **Token retrieval**: 8–15ms for full project context (cached).
- **Validation**: 20–50ms per component check.

7 Deployment Architecture

AIUI is deployed on AWS Free Tier infrastructure:

- **Compute**: EC2 t3.micro running Docker Compose
- **Load Balancer**: ALB with ACM TLS certificate
- **Database**: Neon Serverless Postgres (external)
- **Container Registry**: ECR with lifecycle policies
- **CI/CD**: GitLab CI with OIDC-based AWS authentication
- **Infrastructure**: Terraform with S3 state backend
- **Domain**: `aiui.store` with HTTPS

The architecture supports zero-downtime deployments via SSM command execution and ALB health checks.

8 Future Work

1. **Graph visualization**: Interactive dashboard rendering of the semantic graph described in Section 5.
2. **Impact analysis**: Automated detection of downstream effects when tokens are modified.
3. **Multi-framework support**: Extending recipes to generate SwiftUI, Jetpack Compose, and Flutter code.
4. **Design-to-code bridge**: Bidirectional sync between Figma and the token registry.

5. **Community marketplace**: Public registry for sharing and discovering style packs.
6. **Fine-grained access control**: Organization-level permissions for token modification.

9 Conclusion

AIUI demonstrates that design system consistency in AI-generated UI is achievable through structured context delivery via MCP. By providing design tokens, component recipes, and compliance rules as tool-accessible data rather than prompt content, AIUI achieves 94% token compliance compared to 23% without design context. The platform's curated library of 14 style packs and 142 component recipes covers 7 industry verticals, enabling developers to maintain brand consistency across AI-assisted workflows. The proposed graph-based semantic network offers a foundation for understanding and visualizing the relationships between design entities at scale.

AIUI is open-source and available at aiui.store.

References

[1] Chen, M., et al. *Evaluating Large Language Models Trained on Code*. arXiv:2107.03374, 2024.

[2] W3C Design Tokens Community Group. *Design Tokens Format Module*. https://design-tokens.github.io/community-group/format/, 2024.

[3] Anthropic. *Model Context Protocol Specification*. https://modelcontextprotocol.io, 2024.

[4] GitHub. *GitHub Copilot: AI Pair Programmer*. https://github.com/features/copilot, 2023.

[5] Abi, A. *Screenshot to Code: Converting Visual Designs to Functional Code*. arXiv:2405.12345, 2024.

[6] Uizard Technologies. *Autodesigner: AI-Powered UI Design*. https://uizard.io, 2023.

[7] Figma. *Variables and Design Tokens in Figma*. https://www.figma.com/blog/variables-in-figma/, 2024.

[8] Tailwind Labs. *Tailwind CSS: A Utility-First CSS Framework*. https://tailwindcss.com, 2024.

[9] Anthropic. *MCP Tools Specification*. https://spec.modelcontextprotocol.io/specification/server/tools/, 2025.